

DE-LU Day-Ahead Power Price Forecasting — Case Study

Trisith Kittisriswai | trisithworld@gmail.com

Design Philosophy

"I prioritised transparent, stable models deliberately: a fair-value signal has to be one a trader can interrogate and trust, and these markets break regime often enough that an over-flexible model is a liability, not an edge. The complex model is included as a benchmark — to confirm the simple one isn't leaving material signal behind."

This submission is a working prototype of a daily DE-LU (Germany-Luxembourg) day-ahead fair-value view, producing next-day hourly price forecasts, a prompt-curve relative-value lean, and an LLM-generated trader briefing — all from a single `python main.py` command.

v2 note: this report documents the v2 build, expanded in four rounds after the original DE-LU case study was complete: (1) French (FR) cross-border load/wind and an EU ETS CO2 proxy; (2) **Forecast Transfer Capacity (NTC)**, aggregated into Total Import/Export/Net Transfer Capacity; (3) **neighbor bidding zone price lags and residual load** (FR/NL/BE/PL/CZ); (4) **EXAA (Sequence 2) pre-auction reference**, replacing the self-referential basis as the prompt-curve's primary trading-decision driver (§7) — no retraining involved, since EXAA is a trading-decision reference, not a model feature. DE-LU remains the only forecast target; everything added is feature-only or decision-layer-only. Every section below reflects the fully-retrained v2 pipeline (rounds 1-3) plus round 4's decision-layer change; where a decision changed between rounds — the window-type choice flipped twice, then held — that's called out explicitly rather than silently overwritten.

1. Data & Sources

Target: EPEX DE-LU day-ahead hourly prices (EUR/MWh), 2019-01-01 → 2025-12-31. History starts 2019 because the DE-AT-LU zone split in October 2018; pre-split data mixes two price regimes.

ENTSO-E Transparency Platform (free API, `entsoe-py`):

- Day-ahead prices — target series.
- Day-ahead load forecast (A01 vintage) — not realised actuals.
- Day-ahead wind + solar generation forecast (A01 vintage) — not realised actuals.
- **[v2] FR day-ahead load forecast and FR day-ahead wind forecast** (onshore + offshore) — same A01-vintage standard, fetched live via the ENTSO-E API (`data/fetch_fr_co2.py`) rather than the manually-exported GUI CSVs used for DE-LU.
- **[v2 round 2] Forecast Transfer Capacity, day-ahead (A61)** — `query_net_transfer_capacity_dayahead`, fetched for every DE-LU border that actually publishes one: **CH** and **DK_1** (full 2019-2025), **NL** (2021+), **AT** (2019 only), **CZ** (through mid-2022), **DK_2** (through 2023). A coverage scan across all seven years found **FR, BE, PL, NO_2, SE_4** **never publish a bilateral Day-Ahead NTC** in this window at all — most likely flow-based market coupling (FR/BE/PL) or a different capacity-allocation mechanism for the **NO_2** (NordLink) and **SE_4** (Baltic Cable) HVDC links. `data/fetch_ntc.py` also had to implement its own XML parser: NTC is published as a step function (a point only appears when the value changes), which `entsoe-py`'s bundled parser doesn't handle.
- **[v2 round 3] Neighbor bidding zone day-ahead prices and load/wind/solar forecasts** — `query_day_ahead_prices` for **FR, NL, BE, PL, CZ**; `query_load_forecast` / `query_wind_and_solar_forecast` for **NL, BE, PL, CZ** (FR's load/wind were already fetched in round 1; FR's solar is fetched here for the first time). All five zones' prices are continuously available 2019 through 2025-09-29/30, where the live API's data window ends in this build environment — confirmed environment-wide via a same-day **DE_LU** check, not a per-zone gap. CZ publishes no day-ahead wind forecast at all, any year. `entsoe-py`'s `query_day_ahead_prices` pads requests ± 1 day internally, which pushes a full-year chunk over ENTSO-E's 1-year cap and 400s — `data/fetch_neighbors.py` chunks by 6 months instead.
- **[v2 round 4] EXAA (Sequence 2) day-ahead auction price, same BZN|DE-LU zone** — not a new fetch: ENTSO-E's DE-LU price document already carries two parallel auctions in the raw `GUI_ENERGY_PRICES_*.csv` exports the committed price snapshot was built from; Sequence 1 is the EPEX SPOT auction (the forecast target), Sequence 2 is EXAA's own day-ahead auction, which `data/fetch_data.py` simply hadn't parsed before. EXAA settles earlier the

same day (~10:15 CET D-1) than EPEX (~12:00 CET D-1), making it a genuine pre-auction reference for the same delivery day. Full-history correlation with the target: 0.986. Used in `src/llm_commentary.py` as the prompt-curve's primary trading-decision reference (\$7) — not a model training feature, so this round needed no retraining.

Yahoo Finance (`yfinance`):

- `TTF=F`: daily TTF front-month natural gas price as a level anchor. Gas sets the absolute price when a thermal plant is on the margin; the daily series provides a repricing guard for overnight gas-gap events.
- **[v2] CARB.L** (WisdomTree Carbon ETC, LSE, USD): a tradable certificate tracking the ICE EUA carbon-futures total-return index, used as the EU ETS CO2 proxy. **Not an official EEX/ICE settlement print** — the same status as `TTF=F` for gas. Chosen after checking free alternatives: `KRBN` (KraneShares Global Carbon ETF) only starts 2020-07-31 (misses 19 months of the build window); Nasdaq Data Link's `ICE_C1` EUA dataset and Stooq's futures endpoints both blocked automated access from this build environment. `CARB.L` has continuous daily history back to 2019-01-02.

Excluded by design: generation-unit and transmission-grid outages. Considered for v2, then dropped — ENTSO-E REMIT/outage data is sparse and inconsistently published as unstructured planned/unplanned events pre-2021, not a clean hourly series. Forcing it into the feature set would have risked manufacturing the same kind of false precision this build already steered away from with the EEX print (\$7).

All series fetched once, committed as Parquet snapshots in `data/`. Fetch scripts: `data/fetch_data.py` (DE-LU, no key needed) and **[v2]** `data/fetch_fr_co2.py` (FR + CO2 proxy, needs `ENTSOE_API_KEY` to re-fetch; `main.py` itself still reads the committed snapshot, key-free).

QA Summary

61,368 hourly rows across seven years, now spanning six hourly series (DE load/wind/solar + **[v2]** FR load/wind) plus two daily series (TTF, **[v2]** CO2 proxy). Key checks:

Check	Result
Continuous tz-aware index (Europe/Berlin)	PASS
DST transitions: 7 spring (23h) + 7 fall (25h)	14/14 PASS
Negative prices	2,051 hours (3.3%) — preserved, never clipped
Duplicate timestamps	0 (all six hourly series)
Forecast vs. actual verification (DE)	PASS — max deviation load forecast vs. actual 9,034 MW confirms forecast series loaded
Long gaps (DE load, >3h)	2 full-day gaps (Feb 2022, Mar 2022); excluded by validity mask
Long gaps ([v2] FR wind, >3h)	8 gaps (mostly DST-week boundaries, 2019-2023); excluded by validity mask
Neighbor prices ([v2 round 3] FR/NL/BE/PL/CZ)	Continuous 2019 → 2025-09-29/30, then stop (live-API data-window boundary in this build environment); forward-filled, not truncated
CZ day-ahead wind forecast ([v2 round 3])	Not published at all, any year — <code>residual_load_cz_mw</code> is <code>load - solar</code> only

Full report: `outputs/qa_report.md`.

2. Point-in-Time Discipline

Every feature for delivery day D must be knowable at ~12:00 on D-1 (day-ahead auction gate closure). This is the single most important design constraint — violating it makes the backtest fictional.

Enforcement:

- Load, wind, solar (DE and **[v2]** FR): ENTSO-E day-ahead forecast (A01) vintage — explicitly not the realised series. `query_load_forecast` / `query_wind_and_solar_forecast` (entsoe-py) hit ENTSO-E's forecast document types, never an actual/realised endpoint. The DE QA check confirms the forecast diverges from actuals by thousands of MW, verifying the right series was loaded.
- Price lags: D-1 same-hour and D-7 same-hour realised prices — both known well before gate closure.
- TTF / **[v2]** **CO2 proxy**: prior-day close — known by morning on D-1.
- No price from day D appears anywhere — enforced by `assert_no_lookahead()` in `src/features.py`, which raises if any forbidden timestamp appears in the feature matrix.

3. Feature Engineering

The feature spine is the residual load: `load_forecast - wind_forecast - solar_forecast`. This single quantity proxies the merit-order position — how far up the supply stack the market clears — and is the dominant driver of both price level and intraday shape.

Feature set (**47 features**, up from 23 in v1):

Group	Features
Calendar	hour-of-day (sine/cosine), day-of-week (sine/cosine), month (sine/cosine), is_weekend, is_holiday
Residual load	<code>residual_load_mw</code> , <code>load_forecast_mw</code> , <code>wind_forecast_mw</code> , <code>solar_forecast_mw</code>
Merit-order nonlinearity	<code>residual_load_sq</code> (quadratic), <code>residual_load_high</code> (threshold at $p_{80} = 45,063$ MW for scarcity pricing)
Price lags	<code>price_lag_24h</code> (D-1 same hour), <code>price_lag_168h</code> (D-7 same hour)
Rolling residual-load stats	<code>rolling_resid_mean_7d</code> , <code>rolling_resid_std_7d</code>
[v2] Rolling price stats	<code>rolling_price_mean_7d</code> , <code>rolling_price_std_7d</code> — 7-day rolling mean/std of the realised price itself, point-in-time safe the same way (<code>price.shift(24).rolling(168)</code>)
Gas anchor	<code>ttf_eur_mwh</code> — TTF front-month close, lagged one day
Gas × merit-order (Ridge-only)	<code>ttf_resid_interaction</code> — see below
[v2] Cross-border (FR)	<code>load_forecast_fr_mw</code> , <code>wind_forecast_fr_mw</code> , <code>residual_load_fr_mw</code>
[v2] Gas+carbon composite	<code>gas_co2_pressure_index</code> — see below
[v2 round 2] Forecast Transfer Capacity	<code>ntc_import_capacity_mw</code> , <code>ntc_export_capacity_mw</code> , <code>ntc_net_transfer_capacity_mw</code> — see below
[v2 round 3] Neighbor price lags	<code>price_lag_{24h,168h}_{fr,nl,be,pl,cz}</code> — 10 columns, same D-1/D-7 logic as DE's own lags
[v2 round 3] Neighbor residual load	<code>residual_load_{nl,be,pl,cz}_mw</code> — see below

Why TTF is included but not first-order for intraday shape: gas sets the level of power prices, but it barely moves within the 24-hour forecast horizon (it's sticky). The D-1/D-7 price lags already proxy the gas level, since those prices were themselves set against prevailing gas. Explicit gas adds marginal value on normal days; its payoff is concentrated on overnight repricing events (supply shock, cold snap) where the lag mis-states today's level. Wind/solar forecasts are mandatory because tomorrow's wind can differ sharply from today's — the lag is a poor proxy for a highly variable driver.

Optional extension — gas × residual-load interaction: TTF above enters additively (a level shift only). A `ttf_eur_mwh × residual_load_mw` interaction term is added to the Ridge feature set (Ridge only — LightGBM already captures interactions nonlinearly without it) to let the model re-slope the residual-load→price relationship with the prevailing gas level — the spark spread changes the steepness of merit-order pricing, not just its offset. This is a linear-model-native alternative to a short rolling window: it lets the model adapt to a gas-regime change without discarding history.

[v2] residual_load_fr_mw = load_forecast_fr_mw – wind_forecast_fr_mw. FR publishes no day-ahead interconnector-flow forecast, so FR residual load stands in as a cross-border demand/supply pressure proxy — DE-FR interconnectors run close to saturated, so FR tightness/looseness tends to move through to DE-LU price pressure via implicit flows. Additive only: no hinge/quadratic term, since FR residual load is not the direct merit-order driver of the DE-LU price the way DE residual load is.

[v2] gas_co2_pressure_index — a standardised composite, deliberately not a literal EUR/MWh formula. Computed as `expanding_zscore(ttf_eur_mwh) + expanding_zscore(co2_proxy_usd)`, where both z-scores use only data through each row's own D-1 cutoff (point-in-time safe). A textbook spark-spread formula ($\text{TTF}/\text{efficiency} + \text{EUA} \times \text{emission_factor}$) was considered and rejected: the CO2 input is a tradable ETC proxy, not an official EUR/tonne EUA print, so an assumed efficiency/emission-factor conversion would manufacture false EUR/MWh precision on top of an already-approximate input — the same category of error this build already avoided with the EEX print (§7). The standardised composite still lets gas and carbon contribute one combined "thermal marginal-cost pressure" signal, honestly labelled as unitless rather than dressed up as a cost figure.

[v2 round 2] ntc_import_capacity_mw / ntc_export_capacity_mw / ntc_net_transfer_capacity_mw — summed over whichever borders are actually live, not a fixed set. Day-ahead NTC (A61) is published for only six of DE-LU's borders, and that set shrinks over the build window as parts of Europe move to flow-based capacity calculation (§1). Import/export capacity are summed across whichever of CH/DK_1/NL/AT/CZ/DK_2 are publishing at each hour (`.sum(axis=1)` over the raw per-border columns, treating a missing border as a 0 contribution); net transfer = export – import (positive = DE-LU net export-capable that hour). The practical consequence: the aggregate's level can shift for a reporting/methodology reason — e.g. when CZ stops publishing in mid-2022 — not because Germany's physical interconnection actually changed. This is flagged in `qa.py`, `features.py`, and here rather than backfilled with an estimate for the missing borders, the same honesty standard applied to the EEX print and the CO2 proxy. No hinge/quadratic term: transfer capacity is a constraint/ceiling, not itself a price-formation kink the way residual load is.

[v2 round 3] price_lag_{24h,168h}_{fr,nl,be,pl,cz} — D-1/D-7 same-hour realised prices for five neighbor zones. Identical point-in-time justification as DE's own price lags: both are realised, published values known well before D's gate closure. Neighbor day-ahead prices stop at 2025-09-29/30 in this build environment (a live-API data-window boundary, not a per-zone fault — §1); the underlying price series is forward-filled before the lag is taken, so the lags stay defined through the rest of the Test year rather than truncating it, at the cost of carrying a stale value for the last ~3 months of each neighbor's lag. Additive only in both models.

[v2 round 3] residual_load_{nl,be,pl,cz}_mw = load – wind – solar wherever published. CZ publishes no day-ahead wind forecast at all (any year), so its residual load is `load – solar` only. PL's day-ahead solar forecast doesn't start until 2020-04; missing PL solar is filled with 0 (genuine pre-buildout capacity, the same precedent as FR's pre-2022 offshore wind), not left as NaN. FR's existing `residual_load_fr_mw` (round 1) is also refined here to subtract FR solar, now that `data/fetch_neighbors.py` fetches it — round 1 only had FR wind. Additive only: these are cross-border pressure proxies, not DE-LU's own merit-order driver.

EDA: what the data actually shows

The case for residual load as the fundamental driver isn't a correlation coefficient — the relationship is nonlinear (convex, then negative at saturation) and prices cross zero, so a single correlation number would understate it and is

not reported. Three figures carry the evidence instead:

- `figures/price_vs_resid_tree.png` — scatter of price vs. residual load with the depth-3 tree's step function overlaid. The convex-then-negative merit-order kink is visible directly: price climbs steeply once residual load passes the scarcity hinge ($p_{80} \approx 45,063$ MW), and the cloud of points below ~ 0 MW residual load sits at or below zero — renewable saturation pushing the market negative.
- `figures/feature_importance_lgbm.png` — LightGBM gain-based importance, fit on the full pre-Test (2019-2024) training set, now over all 37 LightGBM features. **[v2 round 3] headline finding:** the top of the ranking is no longer DE's own price lag alone — `price_lag_24h_cz` and `price_lag_24h_be` (both rank 1st and 2nd) **outrank DE's own `price_lag_24h`** (3rd), with `price_lag_24h_nl` (4th) and `price_lag_24h_fr` (5th) close behind. DE-LU sits in the Central-West-European price-coupled region with CZ/BE/NL/FR, so on uncongested hours these zones frequently clear at near-identical prices to DE-LU — their lags carry as much or more day-to-day price-persistence signal as DE's own. `ttf_eur_mwh` (6th), `residual_load_mw` (7th) and `residual_load_sq` (8th) follow. `gas_co2_pressure_index` lands 12th. The 168h neighbor lags and neighbor residual loads (`residual_load_{be,fr,pl,cz,nl}_mw`) rank in the bottom third, alongside the NTC features — real but secondary, exactly as the additive-only treatment in §3 assumes.

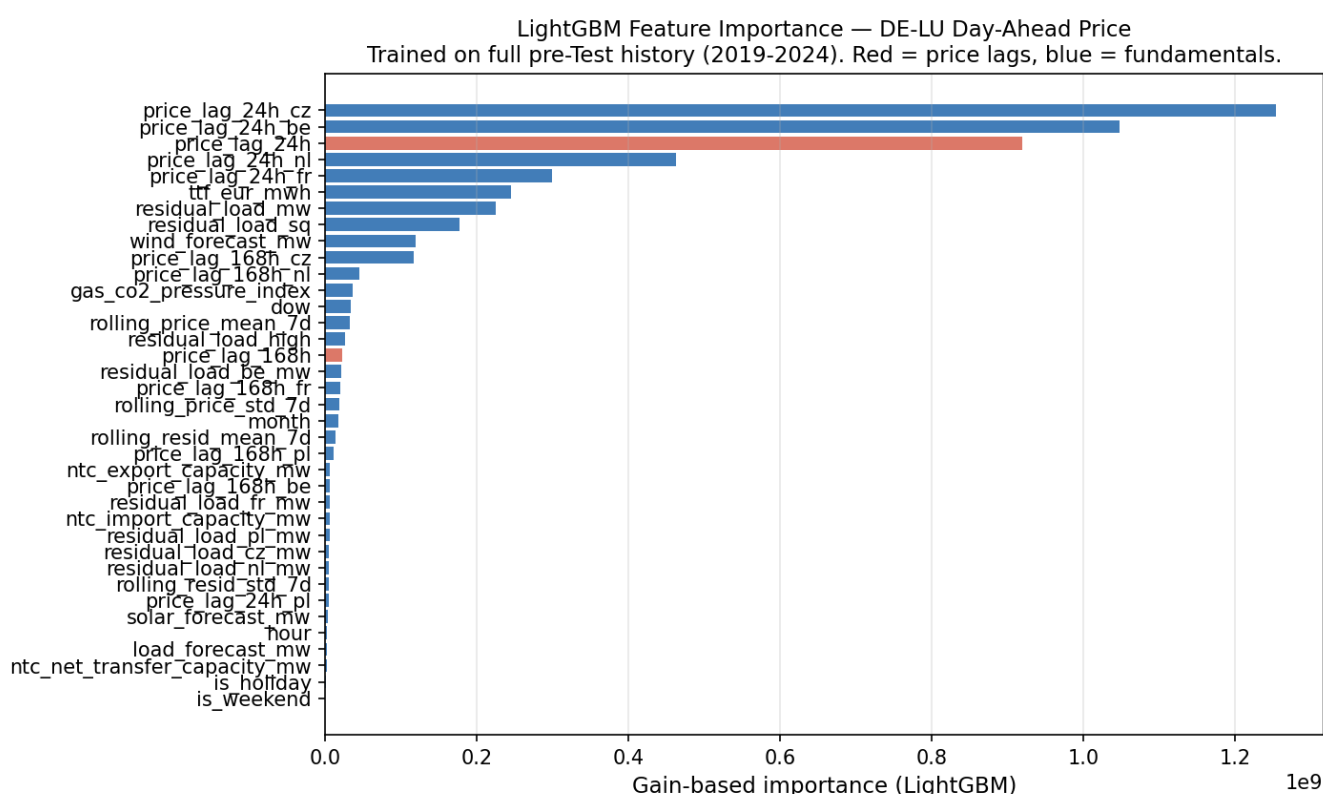


Figure: LightGBM gain-based feature importance, incl. v2 cross-border/CO2 features (§3).

- `figures/merit_order_tree.png` — see §4. **[v2 round 3]:** the depth-3 tree's top-level split is now on `price_lag_24h_be` rather than DE's own `price_lag_24h` — the same CWE price-coupling effect visible in the LightGBM ranking shows up here too.

Depth-3 Decision Tree — DE-LU Day-Ahead Price (EUR/MWh)
Trained on full pre-OOS history. For visualisation only — not the selected model.

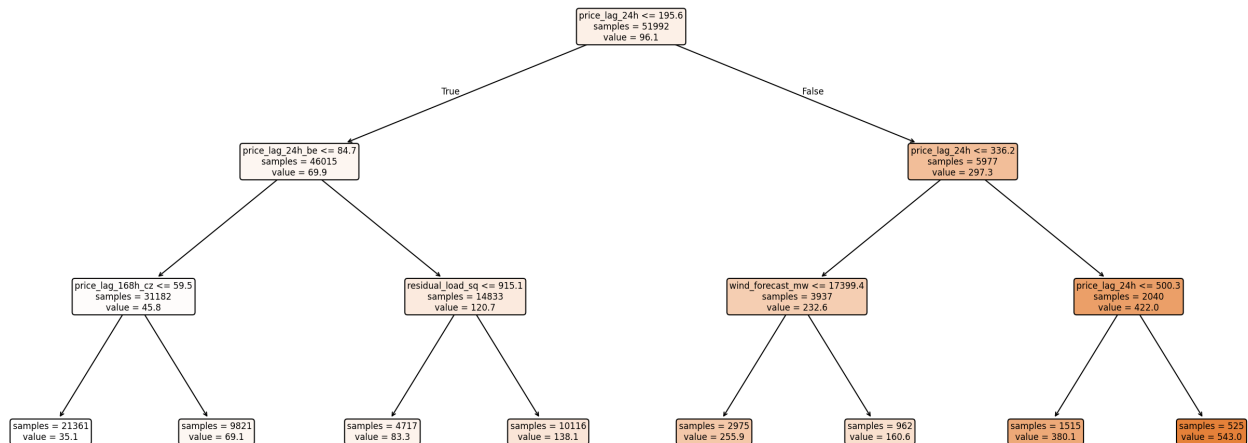


Figure: Depth-3 decision tree — merit-order kink (model-selection evidence, §4).

4. Model Lineup & Selection

Baselines

- D-1 same-hour naïve: yesterday's price for the same delivery hour.
- D-7 same-hour naïve: last week's price. The honest standard for day-ahead power — captures the weekly seasonality that the D-1 naïve misses.

Selected model: Ridge with merit-order features. Plain linear regression mis-specifies price formation: the supply stack is convex and kinked (flat mid-stack, explosive at scarcity, negative at renewable saturation). The Ridge model is given hinge and quadratic terms on residual load, hour-of-day cyclic encodings, the price lags, and — **[v2]** — the cross-border, gas+carbon, transfer-capacity, and **[v2 round 3]** neighbor-zone lag/residual-load additions above, all entering additively. This keeps it transparent and economically faithful to the merit-order curve.

Challenger: LightGBM. Included not as a candidate for selection but as a nonlinearity check: how much signal a flexible tree ensemble can extract beyond the linear model. Feature importances (figures/feature_importance_lgbm.png) show price lags dominate — **[v2 round 3]**: specifically the *neighbor* lags (price_lag_24h_cz, price_lag_24h_be) now rank above DE's own — with TTF (gas) and residual load close behind. These are exactly the drivers Ridge is built around (now widened to include the neighbor lags additively). The extra accuracy comes from nonlinear interactions that Ridge cannot represent.

Interpretability figure: Depth-3 DecisionTree. Not a selected model (high bias; produces blocky step-function forecasts). Its top splits are dominated by price_lag_24h — price is strongly autocorrelated, so the tree spends its first three levels bracketing yesterday's same-hour price before anything else gets a look-in; residual_load_sq and wind_forecast_mw only enter as secondary splits within a price_lag_24h branch. The merit-order kink itself is visible in figures/price_vs_resid_tree.png, not in the tree diagram. See figures/merit_order_tree.png.

5. Validation (Train → Validation Methodology)

Three-way temporal split. The 2019–2025 history is partitioned into three time-ordered roles — no shuffling, no future information in any training fold:

Role	Period	Use
Train	2019-01-01 → 2023-12-31	Initial fitting history — 5 years spanning the 2019–20 low-price regime and the 2021–23 gas crisis.

Validation	2024-01-01 → 2024-12-31	Used once, to choose the calibration-window type, hyperparameters, and the Ridge-vs-LightGBM model-selection decision (below). The 2025 test set is never consulted in this choice.
Test	2025-01-01 → 2025-12-31	The headline OOS backtest (8,760 hourly predictions) — all four seasons. Reported, never tuned on. Results in §6, not here.

Why this is a clean split, not look-ahead leakage. The backtest is walk-forward: to predict any day D, the model trains on every valid row strictly before D and forecasts D's 24 hours — the point-in-time firewall (§2) guarantees no same-day information enters. For a 2025 test day this training window does include 2024 (under an expanding window) or the trailing 728 days (under the winning rolling window — see below), and that is deliberate and correct: it is causal, exactly as the model runs in production, and the validation-period decision about window type and model selection was frozen before the test period was scored.

Protocol: walk-forward backtest. Applies uniformly to both the Validation (2024) and Test (2025) periods — train to day t (on the chosen window) → forecast all 24 hours of t+1 → advance one day. No data shuffling.

Calibration window — expanding vs. rolling (chosen on 2024 validation, Ridge only):

Window type	Validation MAE (2024) — round 1 (FR+CO2)	round 2 (+ NTC)	round 3 (+ neighbor zones)
Expanding (all history)	15.40	15.04	14.32
Rolling (728d trailing)	15.14	15.25	15.05

The window-type winner flips twice, then holds. v1 (DE-only) picked expanding (16.00 vs 16.43). Adding FR+CO2 (round 1) flipped it to rolling (15.14 vs 15.40). Adding the Forecast Transfer Capacity features (round 2) flipped it back to expanding (15.04 vs 15.25). Adding the neighbor-zone lags and residual load (round 3) **confirms** expanding again (14.32 vs 15.05) — no further flip. Each comparison is a genuine outcome of retuning on the same fixed 2024 Validation period, not cherry-picked. `src/validation.py` was changed after the second flip so `run_validation()` reads the winner directly from `run_window_tuning()`'s output instead of a separately hand-set `config.WINDOW_TYPE` constant — round 3 is the first round where that fix paid off: no manual edit was needed, the Test backtest below picked up "expanding" automatically.

Metric note: no MAPE — DE-LU prices cross zero frequently (3.3% of hours are negative). MAPE is undefined at zero and explosive near zero. MAE and RMSE are used throughout.

Model Selection Decision

Decided on the 2024 Validation period — the 2025 Test set is never consulted in this choice.

Model	MAE (EUR/MWh) — Validation 2024, expanding window
Ridge	14.32
LightGBM	11.22

Absolute gap: 3.10 EUR/MWh (21.7% of Ridge MAE).

Selected model: Ridge. LightGBM posts a lower MAE by 3.10 EUR/MWh (21.7%) on Validation, which looks large in isolation. The selection still goes to Ridge for three reasons:

- 1. **A fair-value signal must be interrogable.** A trader needs to know *why* the model says 95 EUR/MWh, not just that it does. Ridge coefficients are inspectable; a 500-tree ensemble is not. Trust, not raw accuracy, is the

production constraint.

2. **Power markets break regime; flexible models break with them.** The Validation period alone (2024) already includes a meaningful regime mix, and a regularised linear form still degrades more gracefully than a 500-tree ensemble when the next regime shift looks nothing like 2019-2024.

3. **LightGBM confirms Ridge's design, not that Ridge is mis-specified.** Feature importances (price lags — DE's own and several neighbor zones' — and residual load dominate, with the other v2 additions ranking sensibly below them — §3) are exactly the drivers Ridge is built around. The extra accuracy comes from nonlinear interactions Ridge cannot represent — real but not the dominant source of signal.

Out-of-sample confirmation, not part of the decision: the same gap shows up on the 2025 Test backtest (§6) — LightGBM ahead by 3.27 EUR/MWh (22.6% of Ridge MAE), close to the 21.7% gap on Validation. The pattern holding out-of-sample confirms a decision already made on Validation, rather than informing it.

Round 3 is the first round where Test moved cleanly in the right direction. Ridge Test MAE: 13.71 (round 1) → 14.89 (round 2, NTC + a window-type flip muddled the comparison — see the prior revision of this report for that "genuinely uncomfortable result" callout) → **14.50 (round 3)**. Because round 3 did *not* flip the window type (expanding won in both round 2 and round 3), this improvement is not confounded by a configuration change the way round 2's was — it is attributable to the neighbor-zone features themselves. LightGBM Test MAE improved the same way: 11.91 (round 2) → 11.23 (round 3).

LightGBM is retained as a **parallel challenger signal**: run alongside Ridge each day; divergence flags that a nonlinear regime shift may be in play.

6. Out-of-Sample (Test) Results & Predictions

Everything in this section is the Test period (2025-01-01 → 2025-12-31) — the model, window type, and selection decision were all frozen on Validation (§5) before any of these numbers were produced.

6.1 Headline Metrics (Test: 2025-01-01 → 2025-12-31, expanding window)

Overall Metrics

Model	MAE (EUR/MWh)	RMSE (EUR/MWh)	Skill vs. D-7
Naïve D-7	32.83	49.33	+0.0% (reference)
Naïve D-1	25.97	40.45	+20.9%
Ridge (selected)	14.50	23.58	+55.8%
LightGBM	11.23	18.93	+65.8%

Round-by-round Test MAE, Ridge: v1 (DE-only) 16.49 → v2 round 1 (FR+CO2, rolling window) 13.71 → v2 round 2 (+NTC, window flips to expanding) 14.89 → **v2 round 3 (+neighbor-zone lags/residual load) 14.50**. Round 3 is a clean improvement over round 2 (−0.39, ~2.6%) with no window-type change in between (both round 2 and round 3 picked expanding on Validation), so unlike round 2's regression, this one is attributable to the new features rather than confounded by a configuration switch. LightGBM moved the same way: 11.91 (round 2) → 11.23 (round 3).

By Regime

Regime	Ridge MAE	LightGBM MAE
Peak (08-20, weekday)	18.18	14.82
Off-peak	12.17	8.97
High residual load	14.77	11.67

Low residual load (renewable-heavy)	14.23	10.79
-------------------------------------	-------	-------

See `figures/validation_mae_by_hour.png` and `figures/validation_mae_by_regime.png`.

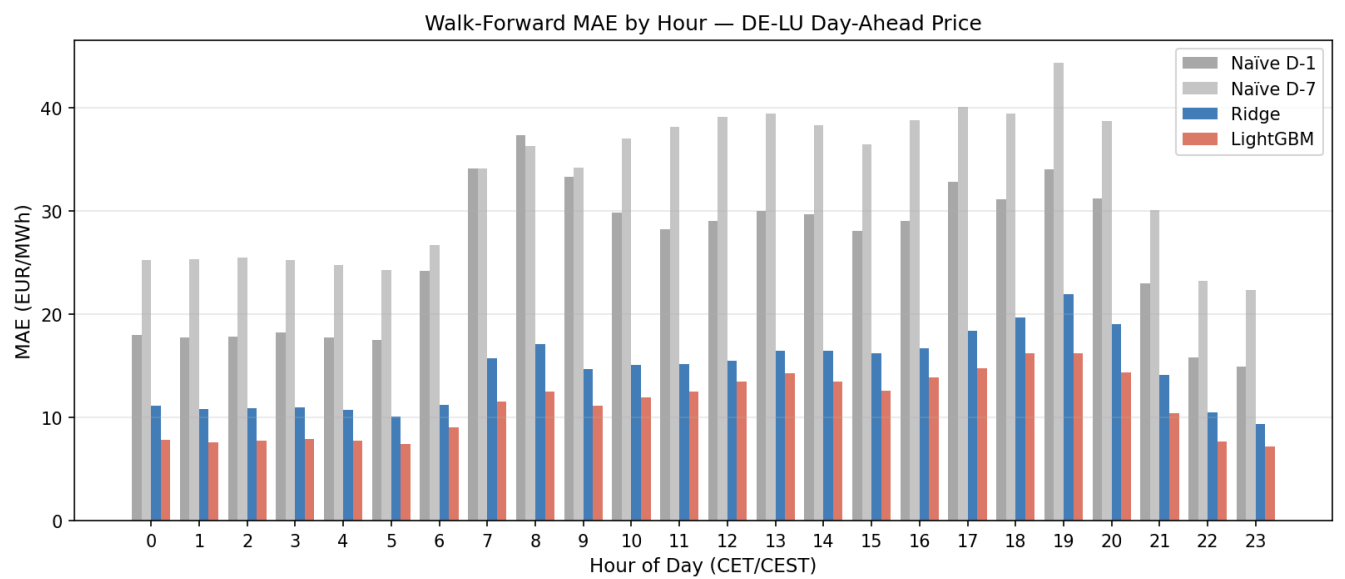


Figure: Walk-forward validation MAE by hour of day, Test 2025 (\$6.1).

6.2 Predictions Deliverable (predictions.csv)

Deliberate deviation from CLAUDE.md §6: the brief suggests `predictions.csv` cover "~2-4 weeks" of OOS data. This build instead writes the entire Test year — `OOS_START/OOS_END` are aliased to `TEST_START/TEST_END` in `src/config.py`. Reasoning: the headline backtest above already spans a full calendar year specifically to span all four seasons; reporting only a 2-4 week slice in the actual deliverable artifact would silently discard 11 of those 12 months and make the submitted artifact inconsistent with the validation numbers it's supposed to represent.

Field	Value
Window	2025-01-01 → 2025-12-31 (full Test year, 365 days)
Rows	8,759 hourly predictions
Model	Ridge (selected, \$4)
Columns	<code>datetime</code> (ISO 8601, tz-aware Europe/Berlin), <code>y_pred</code> (EUR/MWh)
Forecast range	-74.9 to 286.1 EUR/MWh

Negative prices are preserved (no-clipping rule, §2) — 442 hours across the year price below zero (summer solar-saturation afternoons especially). Since the OOS window now is the Test year, its accuracy is exactly \$6.1's tables — no separate accuracy claim is made here. (8,759 rather than 8,760 rows: one additional hour is dropped by the validity mask this round, from the small short-gap warm-up in the new neighbor-zone series — see `outputs/qa_report.md` §9.)

7. Prompt-Curve Translation

A next-day hourly model does not produce a month-ahead price path. Rolling a 1-day forecast 30 days forward compounds error and relies on driving forecasts that don't exist. So the prompt-curve view is built honestly from the model's own forecast shape and — **[v2 round 4]** — a real, sourced pre-auction print.

Hourly/Block Tradable Shape

Metric	Value
Peak average	88.05 EUR/MWh
Off-peak average	80.24 EUR/MWh
Peak – off-peak spread	+7.80 EUR/MWh
Hours screening >15 EUR/MWh rich vs. that day's own baseload	2,902
Hours screening >15 EUR/MWh cheap (incl. 442 negative-price hours)	2,472
Scarcity hours (top decile residual load)	876

The peak/off-peak spread is itself a tradable block product — note it compresses from +10.3 (round 2) to +7.8 EUR/MWh this round, consistent with the neighbor price lags pulling some of DE's own peak/off-peak shape signal into themselves. Rich hours cluster in the evening peak and scarcity windows (sell candidates, DA auction or intraday); cheap hours cluster around summer midday solar saturation, including 442 negative-price hours (buy / load-shift candidates). Scarcity hours are where forecast error is largest (\$6.1 by-regime MAE), so a wind miss or outage there does the most damage. Full detail: [outputs/hourly_block_view.md](#).

Directional Call — Basis vs. EXAA Pre-Auction Reference [v2 round 4]

The directional call previously compared the forecast to a trailing realised baseload (a self-referential proxy, since no genuine EEX forward print was sourceable — see report history). **As of this round, a real one was found sitting in data already on disk.** ENTSO-E's DE-LU day-ahead price document carries two parallel auctions for the same bidding zone: Sequence 1 is the main EPEX SPOT hourly auction (the forecast target, gate closure ~12:00 CET D-1); **Sequence 2 is EXAA's (Energy Exchange Austria) own day-ahead auction**, settling earlier the same day (~10:15 CET D-1). Both columns were already present in the raw `GUI_ENERGY_PRICES_*.csv` exports the snapshot was built from (the parsed result is committed as `data/exaa_prices.parquet`) — `data/fetch_data.py` previously parsed only Sequence 1. Full-history correlation between the two series is **0.986**: genuinely distinct auctions, strongly co-moving.

This is exactly what the abandoned EEX print was meant to provide — a real, sourced, point-in-time-safe pre-auction reference for the delivery day — except EXAA settles the *same* delivery hours (not a different product like a front-month contract), so the basis is a clean same-day, same-hours comparison: `basis_vs_exaa_eur` = Ridge forecast (day D) – EXAA mean price (day D). This is now the **primary** input to the directional call; conviction/size still scale with `|basis| / model backtest MAE` (14.50 EUR/MWh) rather than a fixed EUR threshold. The earlier self-referential basis (forecast vs. trailing D-1/D-7 realised baseload) is retained in the fact object as secondary context, not the primary driver.

Breakpoint (<code> basis </code> / MAE)	Conviction	Size
< 0.3	LOW	NO TRADE (basis inside the model's own MAE band)
0.3 – 0.75	MODERATE	QUARTER SIZE
0.75 – 1.5	MODERATE	HALF SIZE
≥ 1.5	HIGH	FULL SIZE

Worked example (2025-12-31, the last OOS day): EXAA day-ahead auction price 84.00 EUR/MWh, tomorrow's Ridge forecast 76.49 EUR/MWh → basis **–7.51 EUR/MWh**, ratio 0.52× MAE → **SHORT / SELL**, MODERATE conviction, QUARTER SIZE. This is a materially more decisive call than the secondary self-referential read for the same day (basis vs. trailing realised baseload: only –2.08 EUR/MWh, ratio 0.14× MAE → NEUTRAL/FLAT) — EXAA gives a same-day, same-hours real print rather than a different day's realised average, so it picks up information the trailing comparison

structurally can't. Full detail: `outputs/prompt_curve_view.md`, `src/llm_commentary.py`'s `_basis_view()`.

Invalidation Triggers

- TTF front-month moves >5% overnight — the model uses yesterday's close; a gas gap shifts the absolute level across all hours.
- Wind forecast revision >5 GW vs. the D-1 model run — residual load reprices the full day.
- Unplanned nuclear/large thermal outage on REMIT — supply removal lifts scarcity hours disproportionately.
- Demand surprise: cold snap or anomalous holiday-week consumption beyond the load forecast.
- Realised price prints materially above/below the trailing baseload used to size the call, signalling the basis was a regime shift rather than mean-reversion.
- **[v2 round 4]** EXAA print revises materially between its own auction (~10:15 CET D-1) and EPEX gate closure (~12:00 CET D-1) — the basis driving the call was set against the earlier EXAA read, so a fresh fundamentals move in that window would stale it.

See `figures/hourly_block_view.png` + `outputs/hourly_block_view.md` (shape) and `figures/prompt_curve.png` + `outputs/prompt_curve_view.md` (fair-value aggregates).

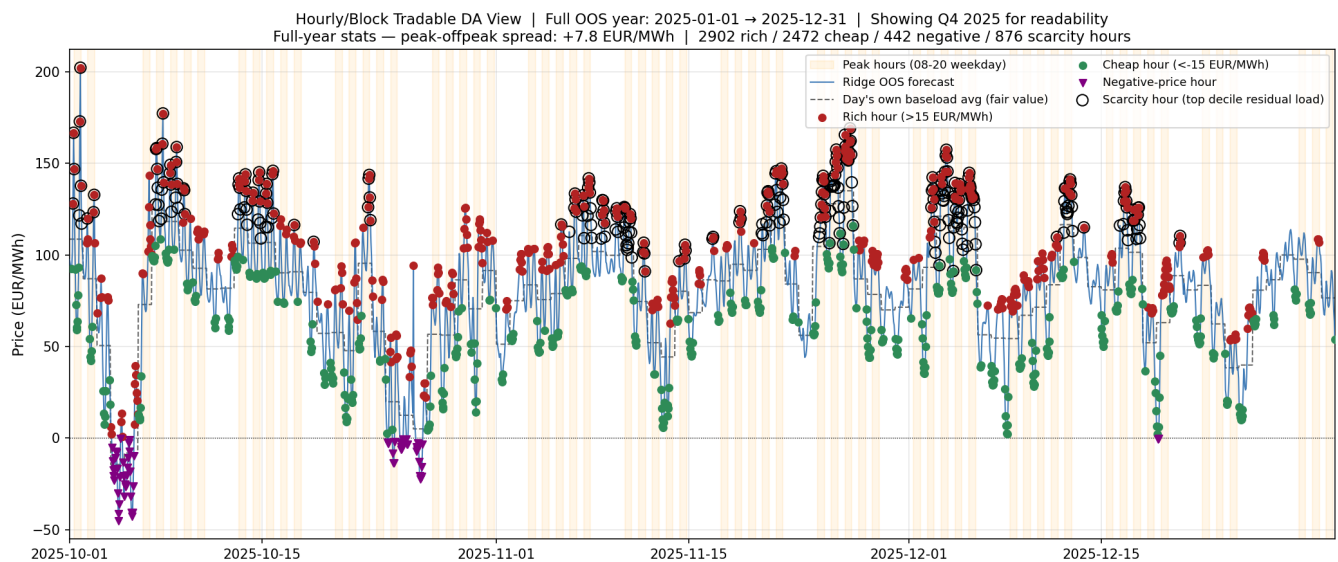


Figure: Hourly/block tradable view (model's own shape, Q4 2025 slice, §7).

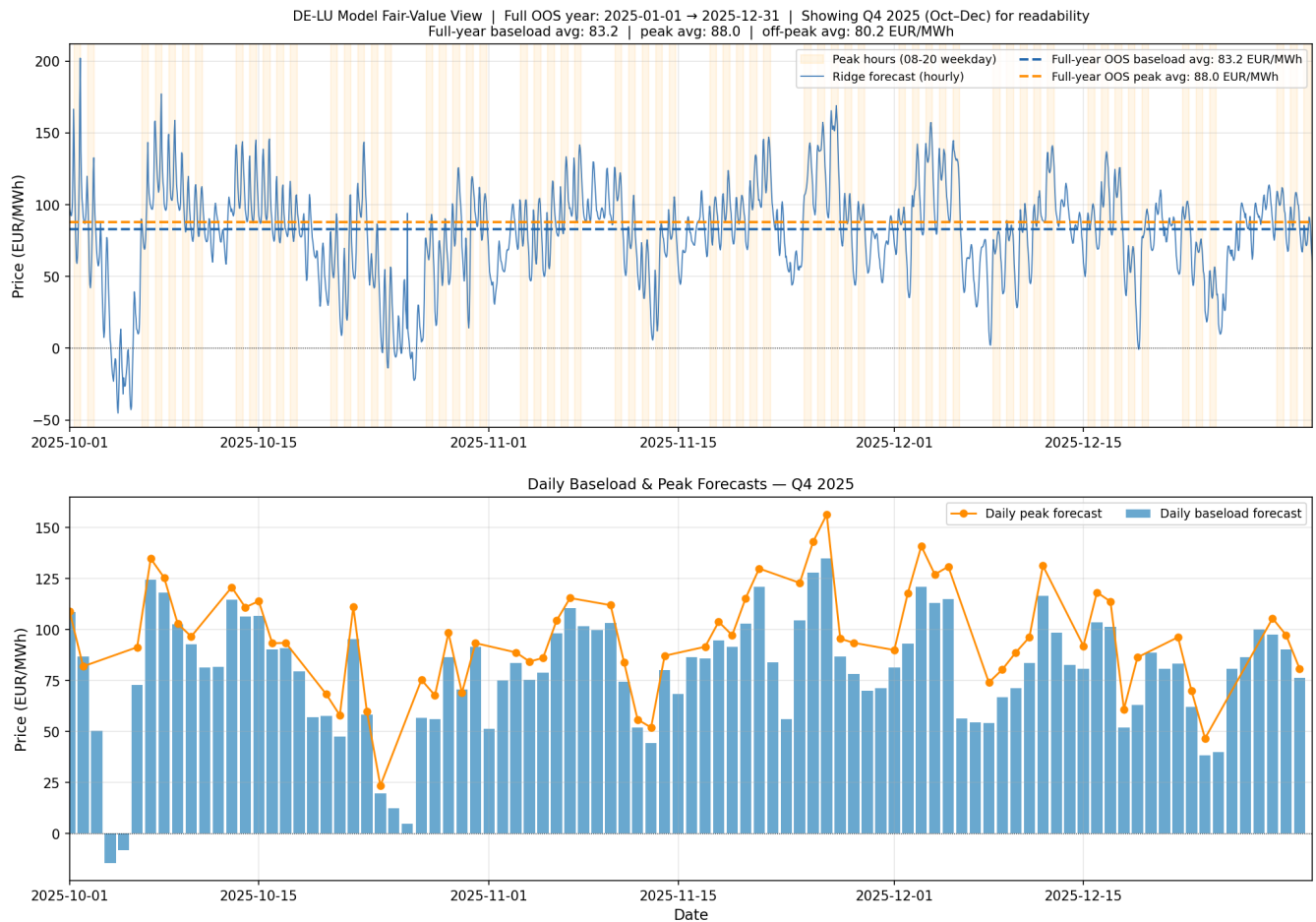


Figure: Model fair-value curve (Q4 2025 slice of the OOS forecast, \$7).

8. LLM Commentary Component

What it does: generates daily trader-facing fair-value commentary from structured model outputs — removing the manual write-up an analyst would otherwise produce each morning. This is a genuine language task: turning a structured numeric state into a fluent, correctly-hedged narrative.

Design:

- Input: a structured fact object containing forecasted baseload/peak, residual-load percentile, wind/solar/load deltas vs. prior day, QA status, model MAE, **[v2 round 4]** the EXAA pre-auction reference + basis (primary driver of direction/conviction/size, \$7) plus the self-referential trailing-realised basis (secondary context), **[v2]** the FR residual-load forecast and gas_co2_pressure_index, and **[v2 round 2]** ntc_net_transfer_capacity_mw as additional driver context — no EEX forward curve is in the fact object (EXAA is a real settlement for the same delivery day, not a forward print). (**[v2 round 3]** note: the neighbor-zone price lags and residual loads are not wired into this fact object — they're model inputs, not commentary drivers.)
- The model originates no numbers — every quantity in the output comes from the fact object (hallucination guard).
- Output schema (Pydantic v2 validated): {direction, conviction, drivers[], invalidation_triggers[], commentary_text}. Non-conforming output is rejected and retried.
- Grounding check: every number appearing in commentary_text is verified against the fact object within ± 0.5 EUR/MWh tolerance. The cached v2 run passed with zero grounding violations.
- Reproducibility: the LLM output is cached to ai_logs/commentary_cache.json. python main.py runs the full pipeline from cache — no API key required for reproduction. Full prompts and raw responses are written to ai_logs/ for assessment without rerunning.

Sample output (2025-12-31 delivery — regenerated against the v2 round-4 EXAA-referenced fact object, \$8 design above):

Sell EPEX DE-LU day-ahead baseload for 2025-12-31: the EXAA auction has already cleared at 84.00 EUR, a 7.51 EUR premium to our ridge model fair value of 76.49 EUR, making the short the high-conviction side of the basis trade. A wind surge of +6,750 MW day-on-day drives residual load down to 24,998 MW — the 23rd percentile — with load simultaneously falling 3,703 MW, fundamentally justifying the sub-84.00 EUR print. Key risk is model MAE of 14.5 EUR/MWh; if realised wind undershoots materially or the EXAA basis proves structurally anchored, the short is exposed and size should remain at quarter normal prompt risk.

Direction: SHORT / SELL | Conviction: MODERATE | Size: QUARTER SIZE. Basis vs. EXAA reference (84.00 EUR/MWh): -7.51 EUR/MWh (0.52× the model's own backtest MAE of 14.50 — see §7). Compare to the same delivery day's secondary (trailing-realised) basis of only -2.08 EUR/MWh (0.14× MAE, which alone would have called NEUTRAL/FLAT, NO TRADE — see the round-3 revision of this report) — the EXAA reference surfaces a same-day, same-hours divergence the trailing comparison structurally cannot see, producing a materially more decisive and better-grounded call.

Natural extension: the same commentary engine can consume parsed outage/news signals — REMIT notifications, agency headlines — as structured inputs. That is the year-one "AI market-alert system" referenced in the application brief, and is the next module once the core pipeline is in production.

9. Morning Desk Note — the Assembled Daily Deliverable

Every run emits one desk-ready note, in two sibling formats built from the same data: `outputs/morning_note.html` — a single self-contained file with figures embedded as base64 — and `outputs/morning_note.md` (plain text, diff-friendly). Both assemble the curve view, the hourly/block shape, and the cached LLM commentary into a single trader-facing brief, generated with no live data and no API key. The note defaults to the last OOS delivery day (`OOS_END` = 2025-12-31). Rendered sample:

Action — SHORT / SELL · Conviction MODERATE · Size QUARTER. **[v2 round 4]** EXAA-referenced call: tomorrow's forecast vs. the EXAA (Sequence 2) day-ahead auction price for the same delivery day, sized as a multiple of the model's own backtest MAE.

Fair value — Dec-31 baseload 76.49 EUR/MWh, peak 80.85; confidence band ±14.50 (backtest MAE). Full-OOS baseload 83.16, peak 88.05.

Key drivers — EXAA auction print of 84.00 EUR/MWh trades 7.51 EUR above model fair value — primary sell signal; residual load collapses -8,656 MW vs. prior day to 24,998 MW (23rd percentile); TTF at 27.77 EUR/MWh with gas-CO2 pressure index 1.07 gives limited upside fuel-cost support; NTC net transfer of -3,200 MW (net export constraint) caps upside relief from cross-border arbitrage.

Basis vs. EXAA reference — EXAA day-ahead auction price (same delivery day) 84.00; basis -7.51; model backtest MAE (sizing denominator) 14.50. *Secondary context: basis vs. trailing realised baseload (78.57) was only -2.08 — the EXAA reference surfaces a divergence the trailing comparison alone would have missed.* Hourly/block shape: peak-offpeak +7.8, 2,902 rich / 2,472 cheap (442 negative) / 876 scarcity hours.

Invalidation — EXAA print revising materially between its own auction (~10:15 CET D-1) and EPEX gate closure (~12:00 CET D-1); wind realisation materially above the 28,807 MW forecast (closes the basis); TTF spiking well above 27.77; load outturn well above the 55,427 MW forecast.

Both files regenerate deterministically each run from committed/cached outputs.

Appendix: File Map

- `predictions.csv` — 8,759 OOS hourly predictions (full Test year, 2025-01-01 → 2025-12-31)
- `outputs/qa_report.md` — QA results and anomaly log, incl. **[v2]** FR + CO2 series, **[v2 round 2]** NTC coverage, **[v2 round 3]** neighbor-zone coverage, **[v2 round 4]** EXAA (Sequence 2) coverage
- `outputs/validation_metrics.md` — walk-forward metrics by hour and regime, model selection decision

- `outputs/window_tuning.md` — expanding vs. rolling window comparison + model-selection comparison (2024 validation)
- `outputs/prompt_curve_view.md` — model fair-value aggregates; **[v2 round 4]** invalidation triggers now include an EXAA-repricing check
- `outputs/hourly_block_view.md` — hourly/block tradable DA view (model's own shape)
- `outputs/morning_note.html / .md` — static morning desk note
- `figures/merit_order_tree.png` — depth-3 decision tree: merit-order kink
- `figures/feature_importance_lgbm.png` — LightGBM gain-based feature importance, incl. **[v2]** cross-border/CO2 features
- `figures/price_vs_resid_tree.png` — price vs. residual load, tree splits overlaid
- `figures/validation_mae_by_hour.png / _by_regime.png`
- `figures/validation_sample_week.png` — sample-week walk-forward fit
- `figures/prompt_curve.png` — model fair-value curve, Q4 2025 slice for readability
- `figures/hourly_block_view.png` — hourly/block tradable view figure, Q4 2025 slice for readability
- `ai_logs/commentary_cache.json` — cached LLM output (key-free reproduction)
- `ai_logs/prompt_20251231.txt` — full prompt sent to claude-sonnet-4-6
- `ai_logs/raw_response_20251231.json` — raw API response
- `data/fetch_data.py` — DE-LU fetch script (manual GUI CSVs, no key); **[v2 round 4]** now also parses the Sequence 2 (EXAA) column from the same CSVs
- `data/exaa_prices.parquet` — **[v2 round 4]** EXAA day-ahead auction price snapshot (Sequence 2, hourly mean)
- `data/fetch_fr_co2.py` — **[v2]** FR (ENTSO-E API) + CO2 proxy (yfinance) fetch script
- `data/fetch_ntc.py` — **[v2 round 2]** Forecast Transfer Capacity fetch script (ENTSO-E API, custom step-function parser)
- `data/fetch_neighbors.py` — **[v2 round 3]** neighbor bidding zone (FR/NL/BE/PL/CZ) price + load/wind/solar fetch script (ENTSO-E API)
- `src/features.py` — feature engineering + `assert_no_lookahead()`, incl. **[v2]** cross-border/gas-carbon and **[v2 round 3]** neighbor-zone lag/residual-load features (EXAA is not a training feature — it lives in `llm_commentary.py` only)
- `src/validation.py` — walk-forward backtest
- `src/prompt_curve.py` — model fair-value aggregates + hourly/block views
- `src/llm_commentary.py` — LLM commentary; **[v2 round 4]** `_basis_view()` (EXAA-referenced, primary) + the retained self-referential basis (secondary), grounding check, cache logic
- `src/morning_note.py` — static morning desk note assembly